AID 825 / Visual Recognition

Visual Product Search Engine

GitHub Repository: VR-Final-Project-Repo

Team Members

Harsh Sinha	IMT2023571
Kartikeya Dimri	IMT2023126
Ayush Mishra	IMT2023129
Nipun Verma	IMT2023591

May 15, 2026

Contents

1 Project Overview

1.1 Introduction

The rapid growth of e-commerce platforms has significantly increased the variety and volume of products available online. However, finding visually similar products through traditional keyword-based search remains a major challenge. Most existing search systems rely heavily on manually assigned product titles, tags, and metadata, which often suffer from inconsistent descriptions, missing attributes, and vocabulary mismatch between users and sellers. As a result, users who discover a product through an image frequently struggle to locate similar items using text queries alone.

This project presents a Visual Product Search Engine that enables query-by-image retrieval for fashion products. Instead of relying only on textual input, the system allows users to upload an image of a clothing item and retrieves visually and semantically similar products from a large product catalog. The proposed system is built using modern deep learning and computer vision techniques, combining object detection, image captioning, multimodal representation learning, and efficient nearest-neighbor search.

1.2 Pipeline

The proposed Visual Product Search Engine is a multimodal retrieval system designed to search for visually and semantically similar fashion products using an input image. The system combines object detection, semantic captioning, contrastive representation learning, and approximate nearest-neighbor retrieval to perform efficient query-by-image product search.

The complete pipeline is divided into two stages: the **Offline Indexing Pipeline**, which prepares the searchable product database, and the **Online Query Retrieval Pipeline**, which processes user-uploaded query images and retrieves matching products.

1.2.1 Offline Indexing Pipeline

The offline stage prepares the searchable product database by generating embeddings for all catalog images and storing them inside a vector index.

Step 1: Product Detection and Cropping. Each catalog image is first processed using a YOLO-based object detector. The detector identifies apparel regions and removes unnecessary background information. The highest-confidence clothing bounding box is selected, cropped, resized, and normalized before further processing. This step improves retrieval quality by ensuring that embeddings focus primarily on the product rather than surrounding background clutter.

Step 2: Semantic Caption Generation. The cropped product image is passed to a BLIP-2 caption generation model. BLIP-2 generates natural language descriptions containing semantic

attributes such as clothing type, color, sleeve style, fit, and texture. These captions provide semantic information that may not be fully captured through visual embeddings alone.

Step 3: Multimodal Embedding Generation. The cropped image and generated caption are encoded using the CLIP model. The visual encoder generates image embeddings, while the text encoder generates caption embeddings. Both embeddings are combined using weighted feature fusion:

$$v_i = \alpha \phi_V(\hat{x}_i) + (1 - \alpha)\phi_T(c_i), \tag{1}$$

where $\phi_V(\hat{x}_i)$ represents the CLIP visual embedding, $\phi_T(c_i)$ represents the CLIP text embedding, and α controls the contribution of image and text features. The final embedding vector is normalized before indexing.

Step 4: Vector Index Construction. All fused embeddings are stored in an HNSW-based Approximate Nearest Neighbor (ANN) index implemented using FAISS. Along with embeddings, associated metadata such as image paths, captions, and item IDs are also stored. HNSW enables efficient large-scale retrieval with low search latency.

1.2.2 Online Query Retrieval Pipeline

The online stage performs retrieval for a user-provided query image.

Step 1: Query Preprocessing. The uploaded query image is passed through the same YOLO detection pipeline used during offline indexing. The detected clothing region is cropped and normalized to maintain consistency between gallery and query representations.

Step 2: Query Embedding Generation. The cropped query image is encoded using the CLIP visual encoder to generate the query embedding vector.

Step 3: Approximate Nearest Neighbor Retrieval. The query embedding is searched against the HNSW vector index using cosine similarity. The system retrieves the top-K nearest product embeddings as candidate matches.

Step 4: Semantic Re-ranking. For improved ranking quality, retrieved candidates are optionally re-ranked using BLIP-2 Image-Text Matching (ITM). The query image is semantically compared against candidate captions, and products with stronger semantic alignment are assigned higher ranks. This improves retrieval precision for visually similar but semantically different products.

Step 5: Final Result Display. The system displays the top-K retrieved products along with similarity scores, captions, and associated metadata through the Streamlit-based user interface. The interface also includes a crop confirmation stage before retrieval to ensure correct localization.

1.2.3 Overall System Flow

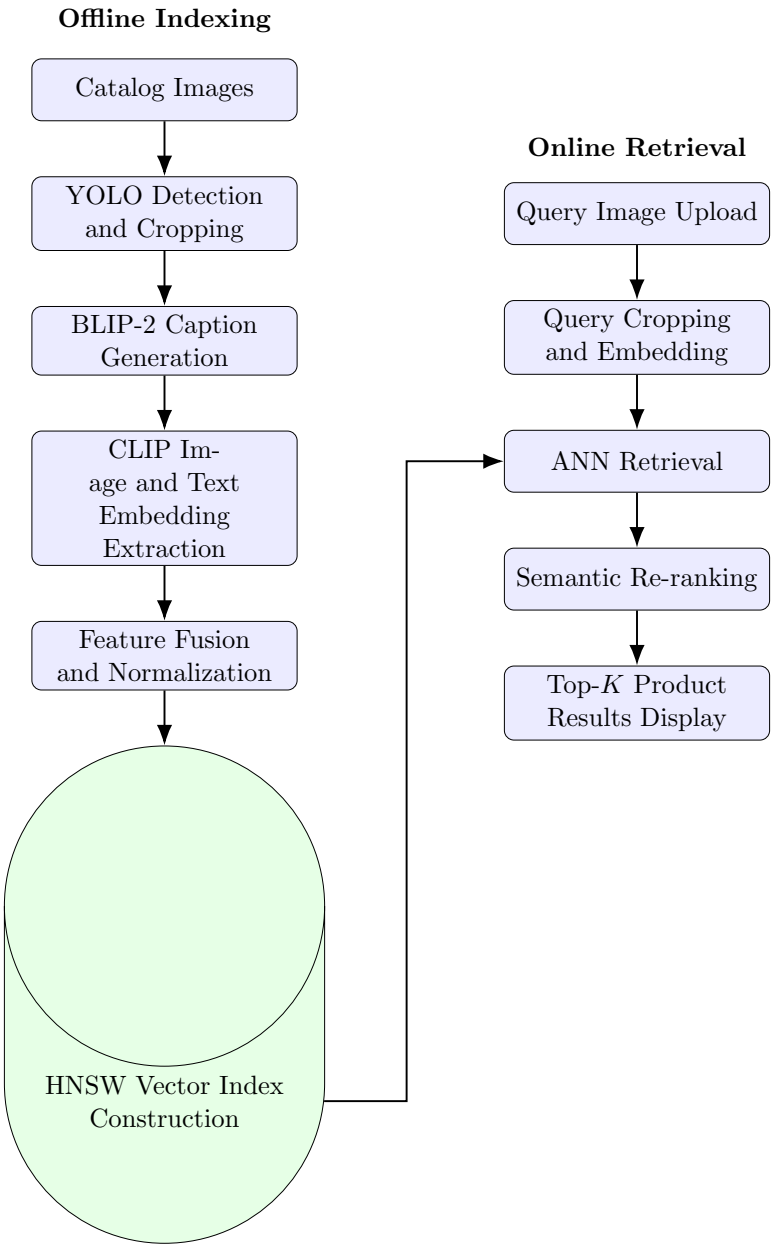

Figure 1: Overall system flow for the Visual Product Search Engine.

2 YOLO-Based Garment Crop Generation

The first stage of the pipeline focused on garment localization and preprocessing using the YOLOv8 framework. In this project, YOLO was used exclusively for generating garment crops from fashion images, which were later used by the downstream captioning, embedding, and retrieval modules.

2.1 Dataset Preparation

The DeepFashion In-Shop Clothes Retrieval dataset was used for crop generation and preprocessing. Bounding-box annotations were obtained from the `list_bbox_inshop.txt` file. Each annotation entry contained:

```
image_name clothes_type pose_type x1 y1 x2 y2
```

The dataset defines three garment-level categories:

- Class 1: Upper-body clothing
- Class 2: Lower-body clothing
- Class 3: Full-body clothing

These labels were remapped into YOLO-compatible class indices:

$$1 \to 0, \quad 2 \to 1, \quad 3 \to 2$$

The annotations were converted into YOLO format:

$$x_c = \frac{x_1 + x_2}{2W}, \quad y_c = \frac{y_1 + y_2}{2H}, \quad w = \frac{x_2 - x_1}{W}, \quad h = \frac{y_2 - y_1}{H}$$

where:

- W, H are image width and height
- (x_1, y_1) and (x_2, y_2) are bounding-box corners
- (x_c, y_c) represent normalized box centers

The dataset was organized into the standard YOLO directory structure:

```
deepfashion_yolo/
    images/train
    images/val
    labels/train
    labels/val
    data.yaml
```

2.2 YOLOv8 Crop Generation Pipeline

A pre-trained YOLOv8n model was used as the garment localization backbone. The model was initialized using COCO-pretrained weights and adapted for garment-region detection on the DeepFashion dataset.

Training was performed on Kaggle using dual NVIDIA T4 GPUs with the following configuration:

The YOLO training code is available in the project repository at YOLO training script.

- Model: YOLOv8n
- Optimizer: AdamW
- Epochs: 20
- Batch Size: 64
- Image Size: 256×256
- Learning Rate: 10^{-3}

The trained detector achieved the following validation performance:

Metric	Value
mAP@50	0.9531
mAP@50:95	0.7780
Precision	0.8813
Recall	0.9198

Table 1: YOLOv8 Garment Detection Performance

The trained weights were exported as:

```
best_deepfashion_yolo.pt
```

2.3 Multi-Class Garment Crop Extraction

After training, a dedicated inference pipeline was developed to generate structured garment crops for downstream retrieval processing.

For each image:

1. YOLO inference was executed.
2. All predicted garment classes were collected.
3. The highest-confidence bounding box for each detected class was retained.
4. Cropped garment regions were saved separately.

The pipeline supported:

- Upper-body crops
- Lower-body crops
- Full-body outfit crops

If a class was not detected for an image, no artificial crop was generated and the corresponding metadata field was left empty.

The crop generation code is available in the project repository at crop generation script.

3 Semantic Caption Generation (BLIP-2)

BLIP-2 (OPT-2.7B) was used to generate a natural language description for each cropped image. The model was loaded in float16 precision with `device_map="auto"` to distribute layers across two T4 GPUs on Kaggle. All weights were kept frozen; BLIP-2 was used purely for inference.

Captions were generated with beam search using 4 beams, a maximum of 40 tokens, and a repetition penalty of 1.2 in batches of 16. A resume-safe pipeline wrote each caption to disk immediately, with periodic `fsync` and full DataFrame checkpoints every 200 batches to handle Kaggle session timeouts.

A total of 55,313 captions were generated across three crop classes with zero nulls:

Crop Class	**Count**
upper_body	34,894
lower_body	11,341
full_body	9,078

Table 2: Caption counts by crop class.

Sample captions from the output are shown below:

Crop	**Caption**
upper_body	a man wearing jeans and a jacket
upper_body	a man in a black and white checkered shirt
upper_body	a man in a grey sweatshirt and blue pants
upper_body	a man in jeans and a blue blazer

Table 3: Sample generated BLIP-2 captions.

Metadata fields such as gender, category, item_id, and shot were then parsed from the structured filename using a __-delimited naming convention, producing a unified `metadata.csv` file with 7,982 unique item IDs passed to the CLIP stage.

4 CLIP Embedding Generation and Fine-Tuning

Model. OpenAI's ViT-B/32 was used through the `open_clip` library. The text encoder was kept fully frozen throughout. For Ablation C, the vision encoder was partially fine-tuned, specifically the last four transformer blocks, the `ln_post` layer, and the projection head, leaving earlier layers frozen.

Fine-tuning setup.

Hyperparameter	Value
Optimizer	AdamW
Learning rate	1×10^{-5}
Weight decay	0.01
Epochs	5
Batch size	64
LR schedule	CosineAnnealingLR
Gradient clipping	max_norm = 1.0
Mixed precision	torch.amp.autocast
Temperature	0.07

Table 4: CLIP fine-tuning hyperparameters.

Loss function. A supervised contrastive loss was used. Within each batch, two images are positive pairs if and only if they share the same `item_id` and `crop_class`. For each anchor, the loss pulls its positives closer while pushing all other items away:

$$\mathcal{L} = -\frac{1}{|P|} \sum_{(i,j)\in P} \left(\text{sim}(i,j) - \log \sum_{k \neq i} \exp(\text{sim}(i,k)) \right), \tag{2}$$

where sim denotes cosine similarity scaled by temperature $\tau = 0.07$.

Feature fusion. After encoding, image and text embeddings are fused as:

$$v_i = \text{normalize}\left(\alpha \cdot \phi_V(\hat{x}_i) + (1-\alpha) \cdot \phi_T(c_i)\right), \tag{3}$$

with α controlling image–text weighting. For Ablation A ($\alpha = 1.0$), only the image embedding is used.

Embedding averaging. Ablations A and B use fully frozen models with no stochastic training, so their embeddings are deterministic; all four seed runs produce identical outputs with standard deviation equal to zero. For Ablation C, fine-tuning introduces seed-dependent variation in the vision encoder weights. Each seed produced a distinct fine-tuned model, and the resulting four embedding matrices were stacked, averaged, and re-normalized to shape $55{,}313 \times 512$. The per-position standard deviation was saved separately and used for the $\pm$ values in the results table.

5 Interactive Demo Application

5.1 Overview

The interactive demo is implemented as a Streamlit-based online retrieval application for query-by-image fashion search. It operationalizes the offline retrieval pipeline in an interactive setting: a user uploads a fashion image, the system localizes garment regions, the user confirms the selected crop, and the confirmed region is encoded and searched against a gallery of precomputed product embeddings. The application integrates the inference-time components required by the project specification: image upload, garment detection, crop display, user confirmation, query embedding generation, approximate nearest-neighbor (ANN) retrieval, re-ranking, and top-K result presentation with similarity scores and metadata.

The application is an inference and demonstration system rather than a training script. Model training, caption generation, CLIP fine-tuning, and gallery embedding construction are performed offline. The Streamlit application loads the resulting assets and focuses on low-latency query processing, stateful crop confirmation, semantic filtering, and robust retrieval over fixed gallery embeddings. The inference application source code is available at Streamlit inference app.

5.2 Runtime Architecture and Resource Management

Streamlit reruns the Python script from top to bottom whenever a widget interaction changes application state. This execution model is convenient for interactive applications, but direct reinitialization of neural models, embedding matrices, and FAISS indices on every rerun would introduce substantial inference latency. The implementation therefore uses Streamlit caching to separate heavyweight runtime resources from immutable processed datasets.

Persistent heavyweight resources are cached with `@st.cache_resource`. These include the CLIP encoder, the Hugging Face object-detection model, and the FAISS index. Such objects are expensive to construct and are reused across interactions during a server session. Immutable processed datasets are cached with `@st.cache_data`. These include the loaded embedding arrays, gallery metadata after filtering, and category subset mappings derived from metadata. These objects are deterministic data products and do not need to be recomputed when a user changes a retrieval option.

FAISS index caching is especially important. The application builds an `IndexHNSWFlat` graph over the gallery embeddings; reconstructing this graph after every slider, checkbox, or crop-selection event would make the online system unnecessarily slow. By caching model loading, data loading, and index construction independently from user-session state, the application keeps the retrieval pipeline responsive while preserving a consistent gallery embedding space.

5.3 Model and Asset Loading

The deployed application loads the following inference-time assets:

- **Detector:** the deployed garment detector is Hugging Face `valentinafevu/yolos-fashionpedia`, loaded through `AutoImageProcessor` and `AutoModelForObjectDetection`. The deployed application does not load `best_deepfashion_yolo.pt`. Both a custom-trained detector and Hugging Face Fashionpedia-based detectors were evaluated experimentally, but the Hugging Face detector was selected for deployment because it produced more stable garment localization and better inference reliability across varied clothing categories. The detector abstraction is modular: downstream retrieval consumes only cropped garment regions and coarse garment classes, so detector replacement requires minimal downstream changes.
- **Encoder:** the query encoder is CLIP ViT-B/32 loaded using `open_clip` with fine-tuned checkpoint weights from `best\_clip\_model.pt`. A fallback to base CLIP ViT-B/32 pretrained weights is included solely as a defensive measure against checkpoint loading failures.
- **Gallery embeddings:** `embeddingsC7.npy` is loaded as a float32 embedding matrix and L2-normalized at load time. Because the embeddings are normalized, dot products are equivalent to cosine similarity during retrieval and re-ranking.
- **Gallery metadata:** `metadata_with_embeddings.csv` is filtered to rows where `split == ''gallery''`. The `embedding_index` column selects the corresponding rows from `embeddingsC7.npy`, preserving alignment between gallery metadata and gallery embeddings after filtering.
- **ANN index:** the gallery embeddings are added to a FAISS `IndexHNSWFlat` index with $M = 32$, `efConstruction`$= 200$, and `efSearch`$= 64$.
- **Gallery images:** result images are resolved from metadata and loaded from `cropped_products/{crop_class}/{filename}` at display time.

5.4 CLIP Checkpoint Loading Strategy

The function `load_clip_weights(device)` implements a defensive checkpoint-loading strategy. The CLIP ViT-B/32 architecture and preprocessing transforms are first initialized using `open\_clip.create\_model\_and\_transforms("ViT-B-32")`. This step is important because retrieval depends on preprocessing consistency: query images must be transformed in the same way as the images used to generate the offline gallery embeddings. If resizing, normalization, or tensor conversion differs between offline and online stages, query embeddings and gallery embeddings may no longer be directly comparable.

The base CLIP ViT-B/32 initialization therefore provides the architecture and transform pipeline. Fine-tuned weights from `best\_clip\_model.pt` are then loaded, overwriting only the model parameters. The preprocessing transforms remain those returned by the base initialization, keeping the deployed query encoder consistent with the offline embedding-generation procedure.

The helper `_extract_state_dict()` handles common checkpoint formats by checking wrapper keys in priority order: `model_state_dict`, `model`, and `state_dict`. If none of these keys are present, the checkpoint is treated as a raw flat state dictionary. The loader also removes a DataParallel `module.` prefix when all checkpoint keys share that prefix. This allows the same deployment code to load checkpoints saved from different training setups.

Weights are loaded with `strict=False`. This tolerates minor key mismatches, such as missing or extra `logit_scale` entries, while still loading the relevant vision-encoder parameters. Missing and unexpected keys are logged to stdout. A fallback to base CLIP ViT-B/32 pretrained weights is included solely as a defensive measure against checkpoint loading failures, ensuring the application remains executable under all deployment conditions.

5.5 Query Encoding and Multi-Crop Fusion

The function `encode_image(image, use_fusion)` converts the confirmed query crop into a normalized CLIP query embedding. In standard single-crop mode, the selected crop is preprocessed with the cached CLIP transforms, passed through `clip_model.encode_image()`, converted to float32, and L2-normalized.

When multi-crop fusion is enabled, the application performs lightweight test-time augmentation on the query crop. Three variants are generated: the original detected crop, an inward crop that removes a 5% margin from each side, and a brightness-enhanced crop produced with `PIL.ImageEnhance.Brightness` using enhancement factor 1.15. Each variant is encoded independently; the resulting vectors are averaged and then normalized again.

This query-time augmentation improves robustness to background noise near bounding-box boundaries, pose variation inside the crop, and moderate illumination variation. It affects only the query embedding. The gallery embeddings remain fixed offline embeddings, and no FAISS index rebuild is required when multi-crop fusion is toggled.

5.6 Fashion Detection Pipeline

The deployed detector uses fine-grained Fashionpedia labels from `valentinafevu/yolos-fashionpedia`. The application maps these labels into three coarse retrieval classes: `upper_body`, `lower_body`, and `full_body`. Upper-body labels include shirts, tops, sweaters, cardigans, jackets, vests, and coats. Lower-body labels include pants, shorts, and skirts. Full-body labels include dresses, jumpsuits, and capes.

This semantic grouping improves retrieval consistency by converting fine-grained detector outputs into the same coarse garment categories used by the gallery organization and category-aware retrieval logic. The detector stage is therefore treated as semantic region localization rather than generic object detection: its purpose is to produce a garment crop and a retrieval-compatible semantic class.

The function `run_detection(image, forced_cls)` runs the image processor and object-detection model, applies the configured confidence threshold, maps valid Fashionpedia labels to coarse classes, clamps bounding-box coordinates to image bounds, and crops each accepted region from the original image. The deployed threshold is `CONF_THRESHOLD = 0.10`. Valid detections are sorted by confidence. In Auto mode, the system returns the highest-confidence detection for each coarse class, yielding at most one upper-body, one lower-body, and one full-body crop. In forced-class modes, only detections matching the selected class are returned; if no detection exceeds the threshold for that class, an empty list is returned and the interface reports the failure rather than silently switching classes.

5.7 Category-Aware Retrieval

The function `get_category_filter(detected_cls, subsets)` maps the detected coarse class to keyword sets derived from the gallery `crop_class` field. Upper-body queries are matched with keywords such as upper, top, shirt, jacket, blouse, coat, sweater, and hoodie. Lower-body queries are matched with lower, bottom, pants, jeans, skirt, shorts, and trousers. Full-body queries are matched with full, dress, jumpsuit, romper, suit, and outerwear.

The matched metadata indices define a category-compatible gallery subset. When category filtering is enabled, ANN search is restricted to this subset before retrieval candidates are generated. This hierarchical semantic filtering prevents invalid cross-category matches and improves retrieval precision by ensuring that the gallery search space is compatible with the localized query garment class.

5.8 FAISS ANN Retrieval and Re-ranking

The function `retrieve(qemb, top_k, cat_filter, use_reranking, dedup)` performs nearest-neighbor retrieval over normalized gallery embeddings. Full brute-force comparison becomes inefficient as the gallery grows, so the full-gallery retrieval path uses FAISS `IndexHNSWFlat`. HNSW represents embeddings as a navigable graph and searches by traversing promising neighborhoods instead of comparing the query embedding with every gallery embedding.

The implementation oversamples before downstream filtering and re-ranking by requesting `top_k` $\times$ 8 candidates. This provides headroom for exact re-ranking and item-level deduplication. If category filtering is active, a temporary `faiss.IndexFlatIP` is built over the filtered gallery subset and searched; otherwise, the cached HNSW index is searched.

Exact cosine re-ranking is optionally applied after candidate retrieval. Because both query and gallery embeddings are L2-normalized, exact dot-product scoring is equivalent to cosine similarity. The application recomputes exact scores for the retrieved candidates and sorts them again. This corrects approximation errors introduced by HNSW graph traversal while avoiding the cost of full-gallery exact search.

The project specification describes BLIP-2 Image-Text Matching (ITM) as a semantic re-ranking

option. The deployed application does not run BLIP-2 ITM during online inference. ITM would require a cross-attention forward pass between the query image and each candidate caption, which is computationally expensive for real-time interaction and difficult to guarantee under CPU-only or limited-GPU deployment. Exact cosine re-ranking was therefore selected as a lower-latency alternative that improves candidate ordering using already-loaded embeddings.

5.9 Result Deduplication

The gallery may contain multiple images for the same product. Without deduplication, the output list can contain several near-duplicate views of one item, reducing product diversity in the displayed results. When deduplication is enabled, the ranked candidate list is scanned in order and only the highest-scoring entry for each `item_id` is retained. This preserves the strongest available match for each product while increasing the number of distinct products shown to the user.

5.10 Interactive Retrieval Workflow

The application workflow explicitly implements the deliverable requirements described in the project specification:

1. **Image upload.** The user uploads a JPG, PNG, or WEBP fashion image, satisfying the requirement that the demo accept an input image.
2. **Garment detection.** The Fashionpedia detector runs on the uploaded image and maps fine-grained garment predictions to the coarse classes used by the retrieval system.
3. **Crop display.** Detected garment regions are cropped from the original image and displayed as selectable candidates, satisfying the requirement to show the detected product crop.
4. **User confirmation.** The selected crop is stored in `st.session_state`. Retrieval is disabled until the user explicitly confirms the crop; the re-selection path resets the confirmation state and allows another crop to be chosen. This implements the required confirmation gate before retrieval.
5. **Retrieval trigger.** After confirmation, the retrieval button executes query encoding, optional category filtering, ANN search, optional exact cosine re-ranking, and optional deduplication.
6. **Top-K result display.** The application displays up to the selected `top_k` retrieved products. Each result is linked to gallery metadata and includes the retrieved image, cosine similarity score, `item_id`, `crop_class`, and a caption preview. Missing image files are skipped with a warning rather than terminating the retrieval flow.

5.11 Runtime Configuration Controls

The runtime controls expose the main retrieval-system decisions for demonstration and ablation-style inspection. The **Top K** slider controls how many products are displayed. The **Region Mode** control selects Auto mode or forces retrieval to Upper Body, Lower Body, or Full Body. Auto mode considers the best detection per coarse class, while forced modes restrict detection and retrieval to a selected garment class.

The **Multi-Crop Fusion** toggle enables or disables query-time augmentation. The **Category Filter** toggle controls whether semantic filtering restricts the gallery before ANN search. The **Cosine Reranking** toggle controls exact post-ANN re-ranking. The **Deduplicate Items** toggle controls whether repeated `item_id` values are removed from the candidate list. These controls affect the active inference pipeline without changing the stored gallery embeddings or rebuilding offline assets.

5.12 Design Tradeoffs and Deployment Decisions

The deployed system reflects several practical engineering tradeoffs. Although a custom-trained detector was developed and evaluated, the final Streamlit application uses the Hugging Face Fashionpedia detector because it gave more stable garment localization and more reliable behavior across varied clothing categories. This decision was possible because the detector module is abstracted from the retrieval module: the retrieval system only requires a cropped garment image and a coarse semantic class.

Exact cosine re-ranking was selected instead of BLIP-2 ITM re-ranking for deployment. BLIP-2 ITM may provide a richer image-text comparison, but its cross-attention computation for every candidate would increase inference latency and require heavier runtime resources. Cosine re-ranking uses the existing query embedding and gallery embeddings, making it substantially lighter while still correcting ANN approximation errors.

The system also applies augmentation only to the query embedding rather than regenerating gallery embeddings. This avoids expensive offline recomputation and FAISS index rebuilding while improving robustness to crop boundaries, pose variation, and illumination changes. Overall, the implementation prioritizes deployment robustness, modular detector replacement, cached resource management, and a balanced retrieval-quality versus computational-cost tradeoff suitable for an interactive visual retrieval demonstration.

6 Experiments and Results

The evaluation scripts and result files are available in the project repository under the Evaluations directory.

6.1 Evaluation Protocol

Dataset split. The DeepFashion In-Shop Clothes Retrieval dataset was split independently per crop class (upper_body, lower_body, full_body). Items with only a single image were excluded. Of the remaining items, 15% were designated as query items and approximately 15% as gallery-only items, with the rest used for training. For each query item, one image was randomly selected as the query; all other images of that item were placed in the gallery. This ensures every query has at least one ground-truth match in the gallery.

Split	Queries	Gallery
All crop classes combined	1,330	15,144

Table 5: Evaluation split across all crop classes.

Ground truth. Two images are a correct match if and only if they share the same `item_id` within the same crop class.

Metrics. Three standard retrieval metrics are reported at $K \in \{5, 10, 15\}$:

- **Recall@K** — 1 if at least one relevant item appears in the top-K results, and 0 otherwise, averaged over all queries.
- **mAP@K** — Mean Average Precision at K. For each query, AP@K accumulates precision at each relevant hit, normalized by $\min(|R|, K)$, and averaged over all queries.
- **NDCG@K** — Normalized Discounted Cumulative Gain, which rewards relevant items appearing earlier in the ranked list and normalizes by the ideal ranking.

Ablation conditions. Five configurations isolate the contribution of each component:

ID	Configuration	α	CLIP	Captions
A	Vision-only baseline	1.0	Frozen	No
B_05	Frozen CLIP + captions	0.5	Frozen	Yes
B_07	Frozen CLIP + captions	0.7	Frozen	Yes
C_05	Fine-tuned CLIP + captions	0.5	Fine-tuned	Yes
C_07	Fine-tuned CLIP + captions	0.7	Fine-tuned	Yes

Table 6: Ablation configurations.

All conditions use the same HNSW index ($M = 32$, efConstruction = 200, efSearch = 128) and cosine similarity for retrieval. Results for Ablation C are reported as mean ± standard deviation

over four seeds (571, 129, 126, 591). Ablations A and B are deterministic because they use frozen models with no training, so their standard deviation is zero.

6.2 Results

Ablation	Config	Recall@5	mAP@5	NDCG@5
A	$\alpha = 1.0$, frozen	0.4248	0.1394	0.1932
B_05	$\alpha = 0.5$, frozen+cap	0.3910	0.1338	0.1829
B_07	$\alpha = 0.7$, frozen+cap	0.4489	0.1621	0.2186
C_05	$\alpha = 0.5$, ft+cap	0.7169±0.010	0.3491±0.006	0.4304±0.007
C_07	$\alpha = 0.7$**, ft+cap**	**0.7462±0.007**	**0.4067±0.006**	**0.4846±0.006**

Table 7: Retrieval results at $K = 5$.

Ablation	Config	Recall@10	mAP@10	NDCG@10
A	$\alpha = 1.0$, frozen	0.4842	0.1306	0.1931
B_05	$\alpha = 0.5$, frozen+cap	0.4692	0.1306	0.1904
B_07	$\alpha = 0.7$, frozen+cap	0.5218	0.1566	0.2239
C_05	$\alpha = 0.5$, ft+cap	0.7852±0.009	0.3454±0.005	0.4437±0.006
C_07	$\alpha = 0.7$**, ft+cap**	**0.7876±0.005**	**0.3949±0.006**	**0.4876±0.005**

Table 8: Retrieval results at $K = 10$.

Ablation	Config	Recall@15	mAP@15	NDCG@15
A	$\alpha = 1.0$, frozen	0.5143	0.1311	0.1983
B_05	$\alpha = 0.5$, frozen+cap	0.5090	0.1321	0.1977
B_07	$\alpha = 0.7$, frozen+cap	0.5617	0.1586	0.2333
C_05	$\alpha = 0.5$, ft+cap	0.8209±0.004	0.3498±0.005	0.4583±0.005
C_07	$\alpha = 0.7$**, ft+cap**	**0.8141±0.006**	**0.3977±0.006**	**0.4986±0.005**

Table 9: Retrieval results at $K = 15$.

6.3 Analysis

Effect of BLIP-2 captions with frozen CLIP (A vs B). Adding captions with $\alpha = 0.7$ (B_07) improves Recall@5 from 0.4248 to 0.4489 and mAP@5 from 0.1394 to 0.1621, a 16% gain in mAP. However, equal weighting at $\alpha = 0.5$ (B_05) hurts performance below the baseline across all metrics. This indicates that with frozen CLIP, the image and text embedding spaces are not well-aligned; giving captions too much weight dilutes the visual signal rather than enriching it. A 70/30 image-caption split finds the right balance.

Effect of fine-tuning (B_07 vs C_07). Fine-tuning the CLIP vision encoder on DeepFashion is by far the dominant factor. C_07 achieves Recall@5 of 0.7462 compared with 0.4489 for B_07, a 66% relative gain, and mAP@5 increases from 0.1621 to 0.4067. Fine-tuning pulls same-item embeddings closer in the visual space, directly targeting the retrieval objective.

Effect of α in the fine-tuned setting (C_05 vs C_07). After fine-tuning, the vision encoder is better aligned with the DeepFashion retrieval task, making the image signal stronger. C_07

($\alpha = 0.7$) outperforms C_05 ($\alpha = 0.5$) across all K values and all metrics, consistent with the frozen case. The 70/30 split remains the better choice regardless of whether CLIP is frozen or fine-tuned.

Stability across seeds. Ablation C shows low variance across the four seeds, with standard deviation at most 0.010 on Recall and at most 0.006 on mAP and NDCG. This confirms that fine-tuning converges reliably and results are not seed-sensitive.

Best configuration. C_07 is the best-performing model, achieving Recall@15 of 0.8141 and NDCG@15 of 0.4986, and is used as the retrieval backbone in the Streamlit demo.